Urbana Board Reference Manual

11862

The Urbana board is an FPGA-based digital system development platform that brings state-of-the-art hardware design tools to the engineering desktop. The Urbana board is a complete, ready to use, stand-alone system that can host circuits and systems ranging from basic logic designs through RISC-V and Micrcoblaze embedded-core designs. The board is centered around AMD's Spartan-7 FPGA, and includes a 1Gbit DDR3 SDRAM, audio and video ports, a USB host controller, and many other features.

Typical designs implemented on the Urbana board include:

- Basic combinational logic circuits like muxes, decoders, adders, multipliers, etc.;
- Registers, counters, state machines and sequential circuits, bus controllers, etc.;
- IP blocks like pulse-width modulators, VGA controllers, FIFOs, digital filters, etc.;
- Processor cores including RISC-V and Microblaze;
- Hardware/software digital systems.

The Urbana board works seamlessly with AMD's freely downloadable Vivado and Vitis CAD software packages. Vivado provides a full-featured circuit design environment for Verilog, VHDL and/or HLS design entry, and it includes simulation, synthesis, implementation, and hardware debugger (logic analyzer) tools. Vitis supports software design in Assembly and C, and it includes a debugger and built-in terminal program for UART communications. Both tools can program and interact with Urbana's FPGA using the built-in USB programming port, and this same USB port also provides board power, a UART/COM port recognized by Windows and Linux, and a debugging port that drives the GNU software debugger and the hardware debugger.

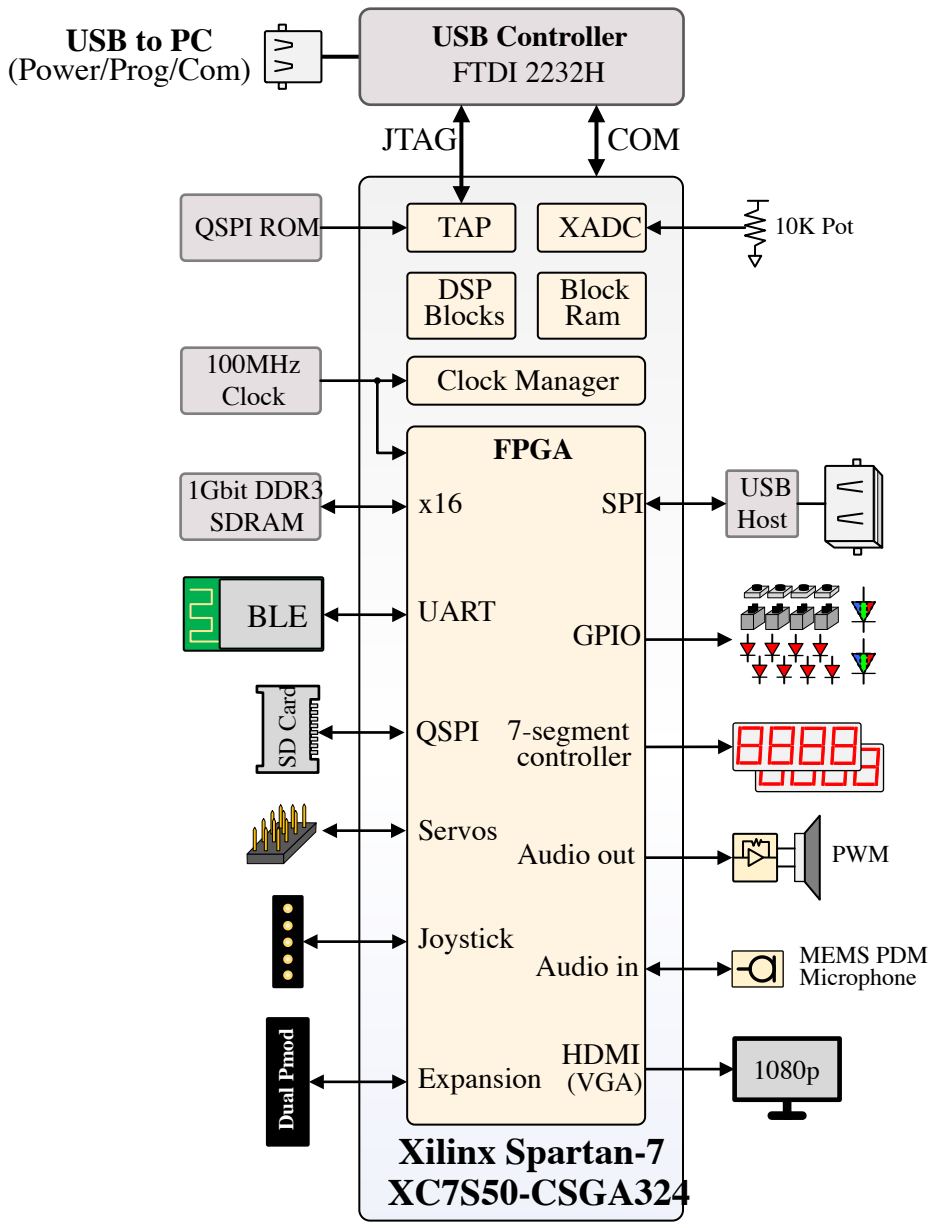
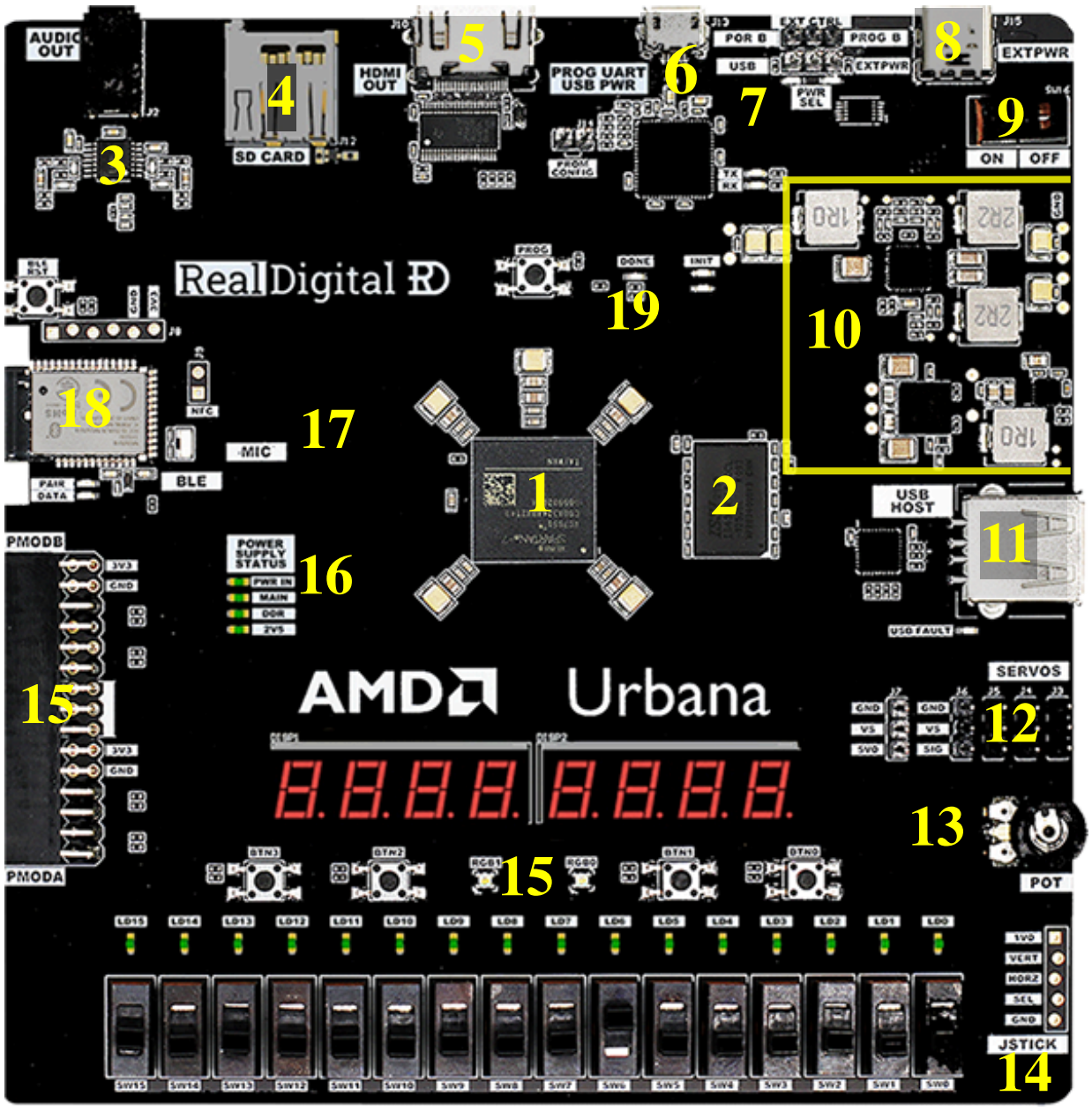


Figure 1. Urbana board diagram

AMD Spartan-7 XCS50 FPGA	32K 6-input LUTs (52K logic cell equivalent)
1Gibt of fast DDR3 SDRAM (x16)	65K+ flip flops & 900Kb of distributed RAM/registers
1080p HDMI source with VGA2HDMI IP block	120 DSP slices with adders, multipliers & accumulators
USB port for power, programming & UART	2.7Mbits of fast, dual-port block RAM
Two Pmod ports, servo and joystick connectors	5 clock management tiles
16 slide switches, 4 pushbuttons, ADC w/pot	Integral dual 12-bit ADC
16 LEDs, 2 RGB LEDs, 8-digit 7seg display	Can host RISC-V and Microblaze embedded processors
Digital MEMs microphone and PWM audio out	Embedded logic analyzer tool for hardware debug
Bluetooth radio and SD card socket	FPGA can be programmed directly from Vivado

Figure 2. Urbana Features



1	Spartan-7 XC7S50-CSGA324 FPGA	10	Power supplies and control
2	1Gbit DDR3 memory (x16)	11	USB host connector and controller
3	Audio output circuit and 1/8" jack	12	Servo motor connectors
4	SD card socket	13	10K Potentiometer (connected to ADC)
5	HDMI source buffer and connector	14	Joystick connector
6	USB power/prog input and FTDI controller	15	GPIO (buttons, switches, LEDs)
7	Power and reset control select	16	Power supply status indicators
8	Optional external power jack (USBC)	17	PDM digital microphone
9	Main power switch	18	FPGA programming status and control

Figure 3. Urbana Board Components

When the Urbana board is first powered on, the FPGA must be configured before the board can do anything. In a typical interactive design environment like a classroom setting, the FPGA is programmed from Vivado. The FPGA can also be configured automatically from an on-board ROM at power-on if jumper block J14 is loaded (see below). The ROM can also be programmed from Vivado. The Urbana board ships with an example design programmed into the ROM, so board function can be verified right out of the box.

Note: A "jumper" is a small, removable connector that can create a connection between adjacent pins - it's the same as a switch, but it's smaller, cheaper, and less likely to be accidently moved from one position to another.

Power Select Jumper J16

Jumper J16 selects whether board power is supplied by the main/programming micro-B USB port or by the type-C USB port - note that the type-C "EXTPWR" USB port supplies only power, and no data signals are connected. Most typically, J16 is set to draw power from the main/programming USB port. In applications requiring more than the 500mA, additional power can be delivered by connecting an external USB power supply using the type C connector and setting J16 to EXTPWR. One of two blue LEDs near J16 will illuminate to indicate which USB connector is providing power.

Programming Jumper J14

Jumper J14 selects whether the FPGA is programmed from Vivado using the USB/JTAG port, or from the on-board SPI ROM. If no jumper is loaded at J14, the FPGA will be held in reset until it is programmed from Vivado over the JTAG/USB programming port. If a jumper is loaded at J14, the FPGA will be automatically configured from the on-board ROM at power-on.

Servo Power Jumper J7

J7 selects the power source for any connected servo motors. If no servo motors are used, then no jumper is needed on J7. If a jumper connects the 5V0 and VS pins, then servo motor power will come from the main board supply (and the main supply can be sourced from either the main/programming USB port or the EXTPWR port as selected by J16). A single servo motor can draw up to 500mA, so when servo motors are being used, it is typical to use EXTPWR, or to use a separate bench supply connected to J7 using a two-pin MTE cable as shown in the figure below.

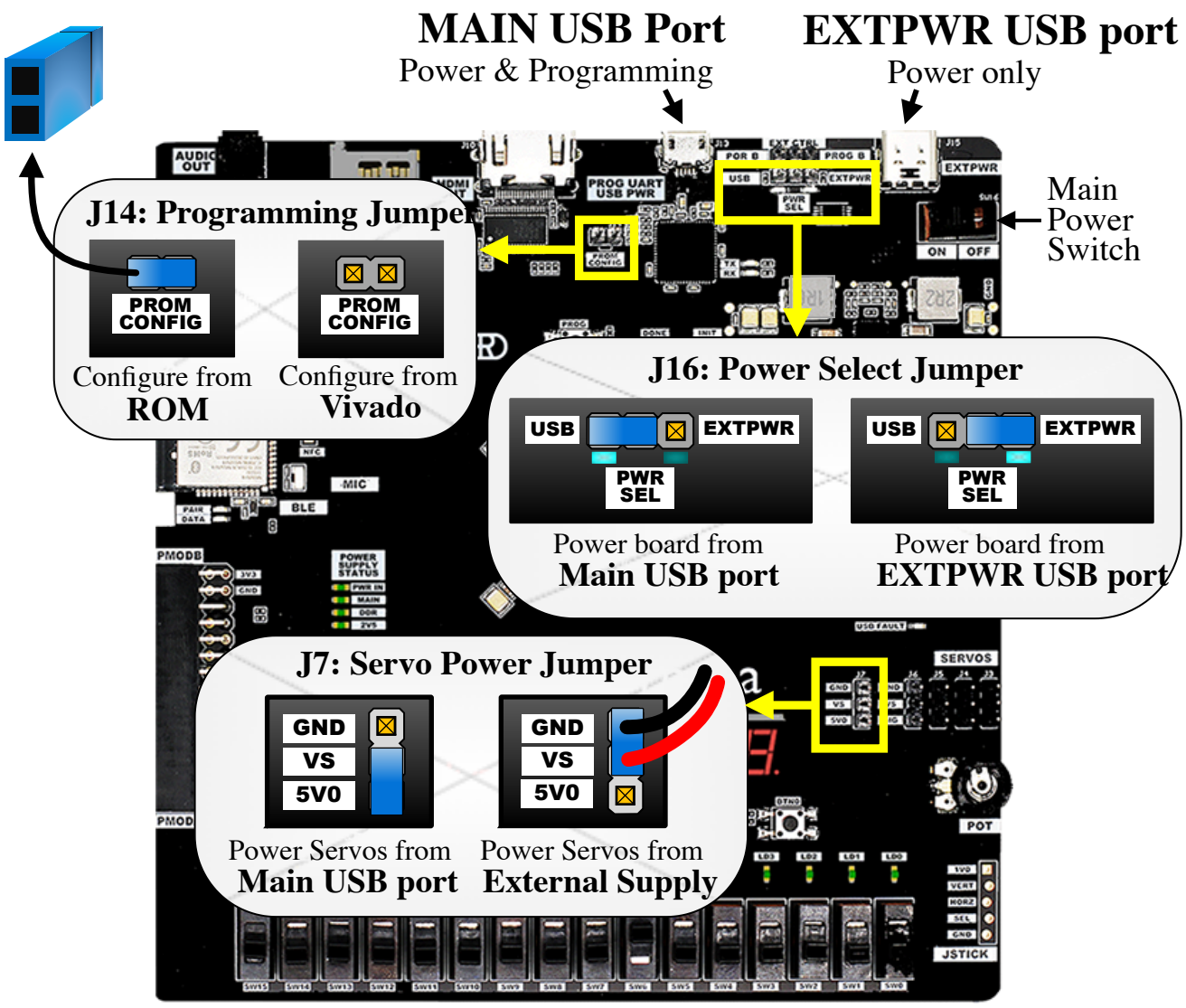


Figure 4. Urbana board jumper blocks and Power Delivery

Power

Overall power consumption is strongly dependent on the FPGA's configuration. Most designs draw less than 500mA, so most designs can be powered from the main/programming USB port. Some designs that drive the FPGA, DDR memory, or data ports at high frequencies, and/or that drive external devices connected to the Pmod or servo ports may consume more than 500mA. In these cases, the EXTPWR USB port can provide additional current.

When the board is powered on and operating normally, a blue LED near the power supply selection jumper (#7 in figure 2 above) will be illuminated to indicate which USB connector is supplying power, and four power supply status LEDs (#17 in figure 2) will be illuminated to indicate the power supplies are operating within normal limits.

Programming

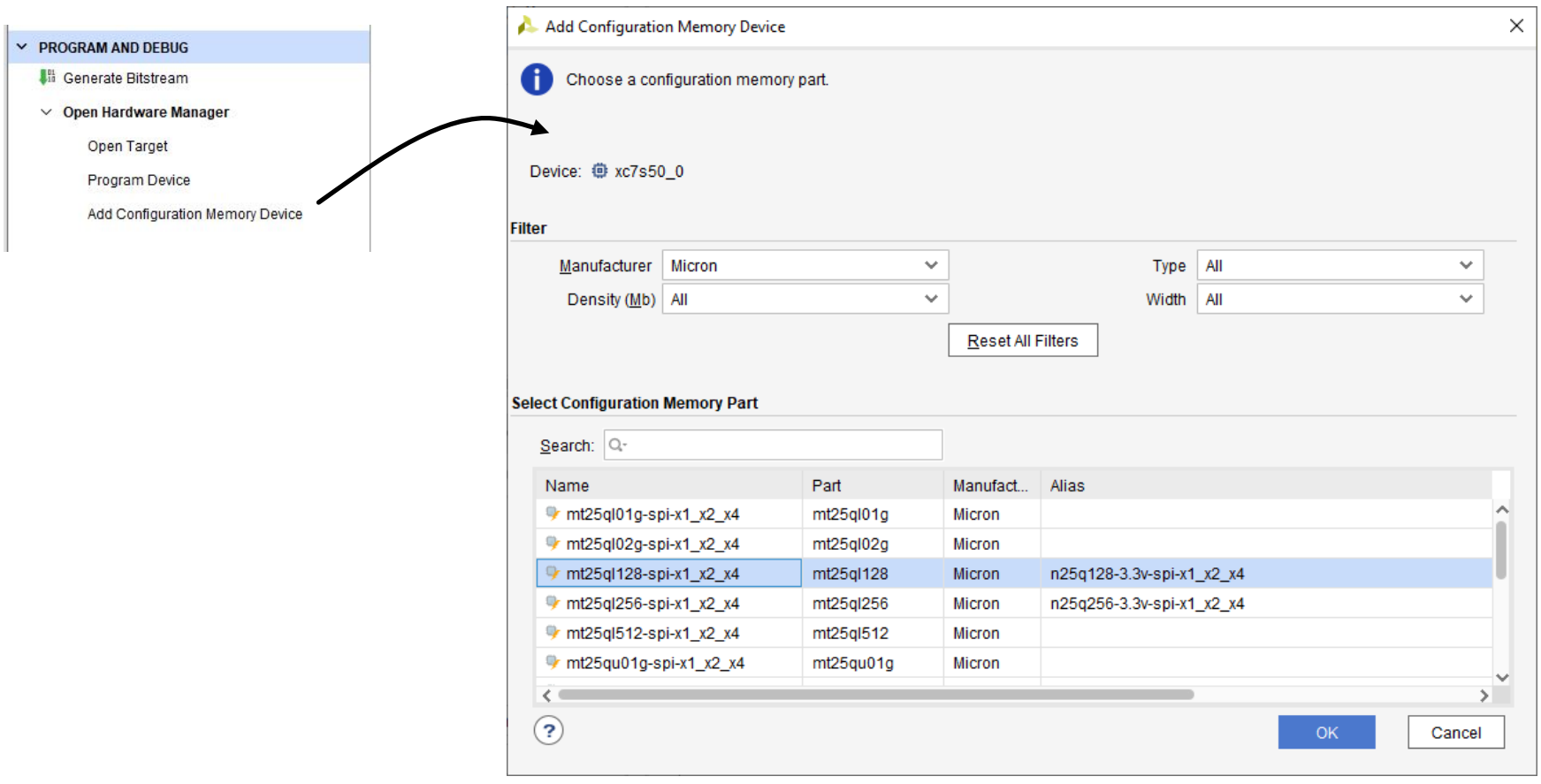
During programming, a ".bit" file is transferred to the FPGA to configure its internal programmable elements. Bit files are produced by Vivado during synthesis, and they can be used to program the FPGA directly, or they can be converted into an "mcs" file that can be used to program the on-board ROM. Jumper J14 directs the FPGA to configure from a live Vivado session or from the on-board ROM. When J14 is absent, the FPGA will be held in reset until it is programmed via the USB/JTAG port from a live Vivado session; when J14 is loaded, the FPGA will be programmed from the on-board ROM. The Hardware Manager tool, which is part of the Vivado tool set, is used to program the FPGA and on-board ROM.

The FPGA must be programmed after the board is powered on, or after the "Prog" pushbutton is pressed. The Prog button, located immediately above the FPGA, resets the FPGA's internal programming memory. When the FPGA is powered on or reset, the INIT Green LED will be illuminated to show the FPGA is not held in reset, is not currently programmed, and is ready to be programmed. The DONE LED will be off until after the FPGA has been programmed successfully, and then illuminated when programming is complete. Both the INIT and DONE LEDs are located near the PROG pushbutton – see the box labelled 20 in figure 3 above. A Red INIT LED indicates an error condition within the FPGA, and if the DONE LED never illuminates, then programming did not occur. These situations could result from several issues – please refer to the AMD documentation (UG470) for more information on error states.

To program the FPGA from a live Vivado session, connect the computer to the Urbana board's programming USB port and power-on the board. From inside Vivado, open the Hardware Manager and select "Open Target -> Autoconnect". The Hardware Manager will scan available ports until the Urbana board is detected, and then display a small panel identifying the xc7s50 FPGA has been detected. To program the FPGA, click on "Program Device" under the Hardware Manager. Note the .bit file from the current project is selected by default, but you can navigate to any other .bit file as well.

To program the FPGA from the on-board ROM, you must first program the ROM in a three-step process: first, add the Micron mt25ql128 ROM to your Vivado project; second, create a .mcs programming file that can be used to program that ROM; and third, use the Hardware Manager to program the mcs file into the ROM (the mcs file contains the same information as the .bit file, but it uses ascii instead of binary codes, and it is the de-facto/preferred ROM programming file type).

To add the ROM to the Vivado project, select "Add Configuration Memory Device", under the Hardware Manager menu on the left side of the Vivado main screen. In the window that pops up, select the mt25ql128-spi-x1_x2_x4 part. Note that you can select "Micron" under the Manufacturer filter pull-down to shorten the list of displayed parts, or just type the part number in the Search window. Click "OK" to add the ROM to the project, and note the ROM part number now appears in the Hardware panel.



Memory Part radio button and verify that the memory part shown is mt25ql128-spi-x1_x2_x4, and enter a filename – be sure to specify the entire path, or else the tools will write the .mcs file to the install directory. Check the "Load bitstream files" box and leave the start address at 0 and the Direction up, and navigate to the bit file (it will be in the "projectname.runs/impl_1" directory). The Write checksum, Disable bit swapping, and Overwrite check boxes can all be left unchecked. Click OK to create the .mcs file.

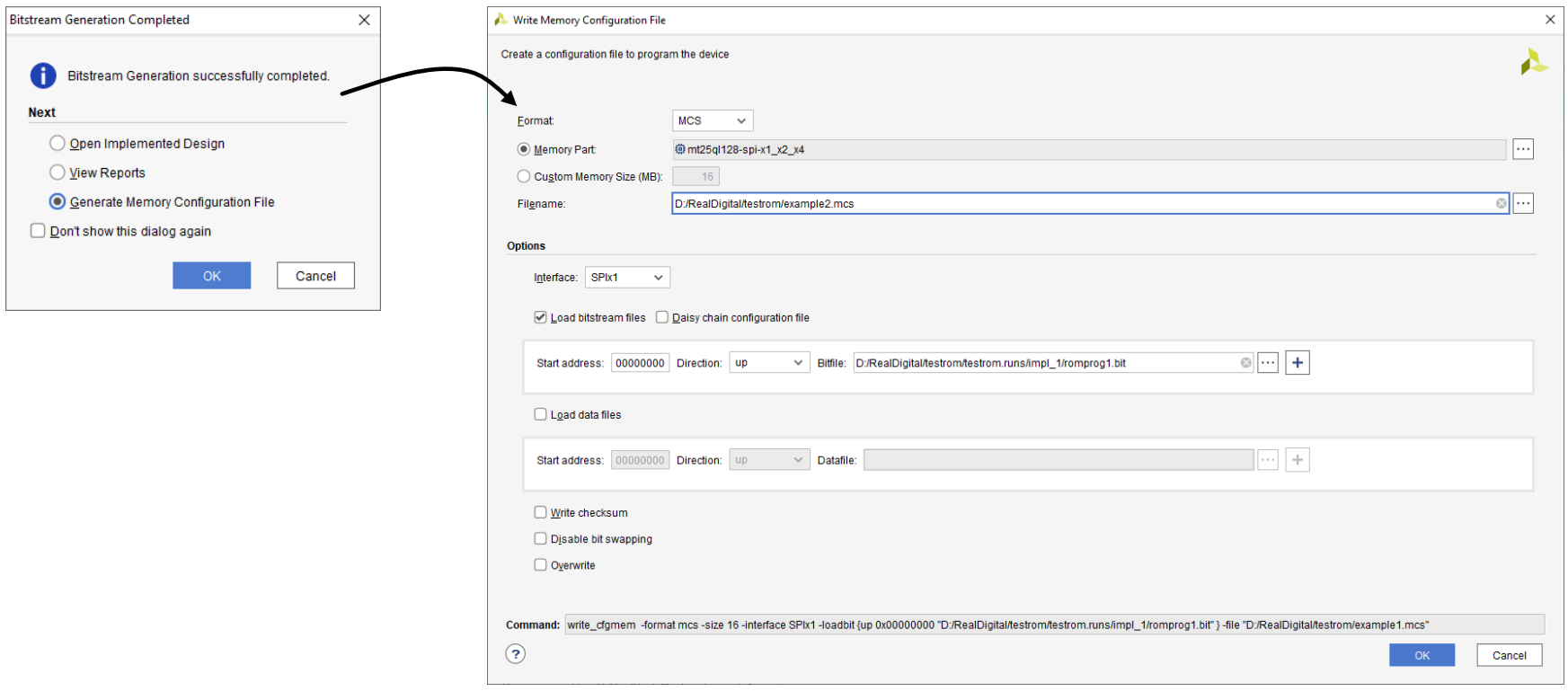


Figure 6. Configuration Memory File Generation

To program the ROM, right-click on the ROM entry in the Hardware panel and select "Program Configuration Memory Device". In the window that opens, you can leave the default selections, navigate to your .mcs file, and click "OK".

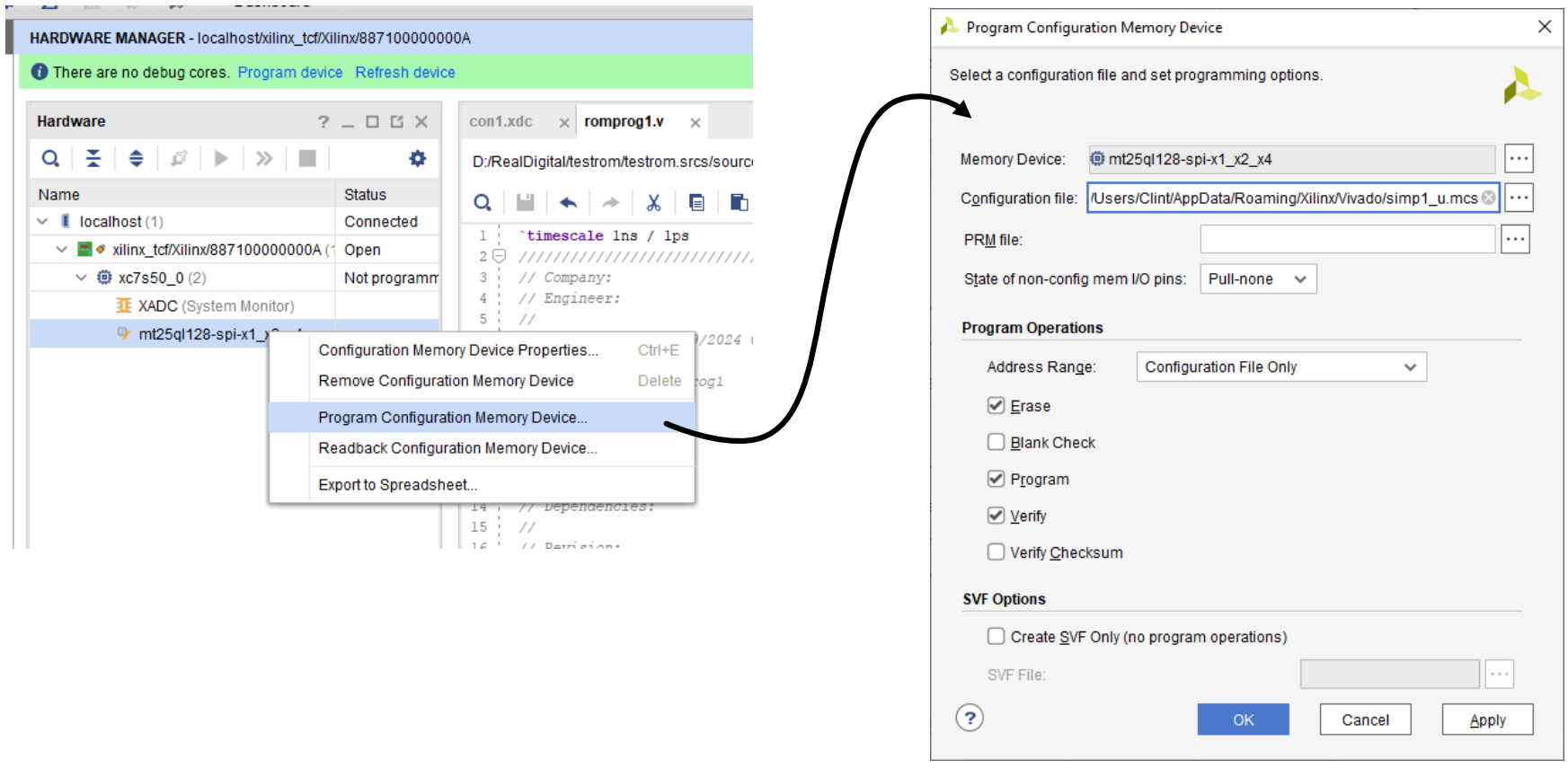
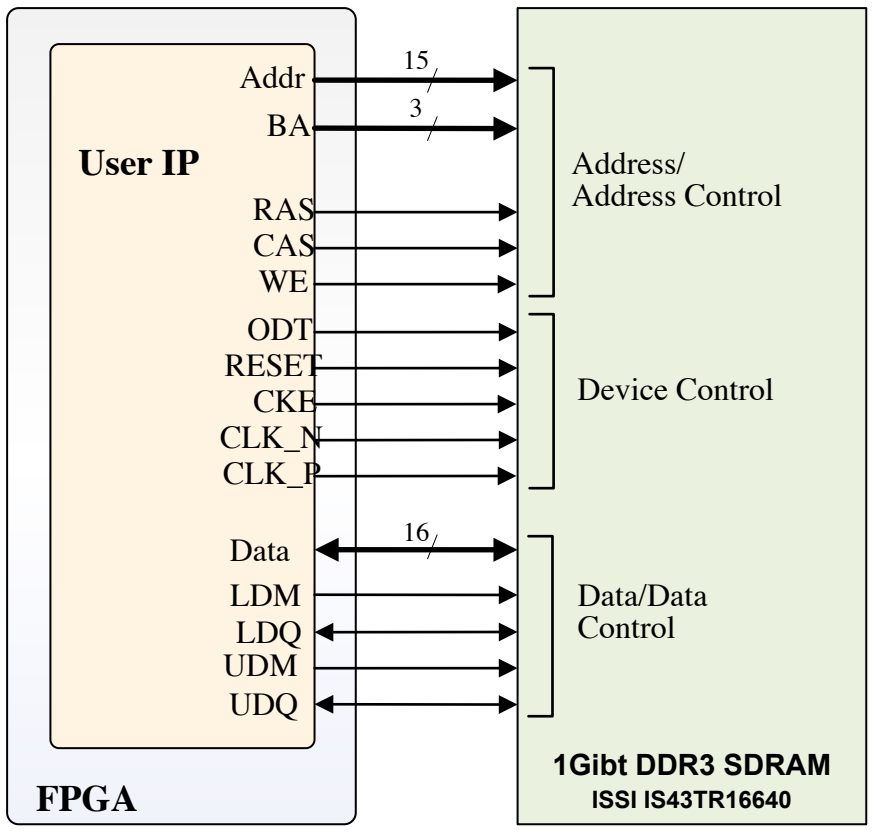


Figure 7. Hardware Manager for ROM programming

If J14 is loaded, a programming file resident in the ROM will be automatically loaded into the FPGA after a power-cycle, or after pressing the PROG button to reset the FPGA. Note that it will take several seconds for the FPGA to program, and the DONE LED will illuminate when programming is complete. Also note that after the FPGA is programmed, you can still program the FPGA directly from Vivado, even if J14 is loaded.

A tutorial and guidelines for producing a Vivado project and source files, and for programming the Urbana board are available here:

matched to within 1mm, they all have unifrom characteristic impedance, and they include impedance-matching 36-ohm series termination resistors on all data signals near the FPGA, and on-die termination in the DDR.



DDR3 Pinout					
Signal	Pin	Signal	Pin	Signal	Pin
Addr0	V3	DQ0	K2	BA0	V3
Addr1	R4	DQ1	M4	BA2	R4
Addr2	P6	DQ2	K3	BA3	P6
Addr3	T3	DQ3	L5	RAS	U2
Addr4	T6	DQ4	L6	CAS	U1
Addr5	T1	DQ5	M6	WE	T2
Addr6	V5	DQ6	L4	ODT	P7
Addr7	U7	DQ7	K6	RESET	M5
Addr8	R7	DQ8	N5	CKE	T5
Addr9	U6	DQ9	M1	CLK_N	T4
Addr10	U3	DQ10	P1	CLK_P	R5
Addr11	P5	DQ11	N1	LDM	K4
Addr12	V6	DQ12	R2	LDQ_N	L1
Addr13	V7	DQ13	N4	LDQ_P	K1
Addr14	R6	DQ14	P2	UDM	M3
		DQ15	M2	UDQ_N	N2
				UDQ_P	N2

Figure 8. Urbana's DDR

Vivado's built-in Memory Interface Generator (MIG) tool can be used to create an IP block that can be added to a Microblaze system, or any other system block diagram. An example project that instantiates a Microblaze processor with DDR memory, a UART, and all required physical constraints is provided in the link below. The design is containted in a Tcl script that can be imported, built, and examined in Vivado to see how the processor core, the AXI bus interface, the MIG and the UART are configured. This same project can serve as a launch point for custom designs.

To import the project into Vivado, download the Tcl script, run Vivado, and use "Tools -> Run Tcl Script" to read and execute the Tcl scipt. Vivado will generate a block diagram and populate all wizard/dialog boxes with configuration parameters; these parameters can be viewed by double-clicking on the individual blocks.

Note: the Tcl file is a text file that was produced in Vivado version 2022.1 – you may need to edit the file and change this entry to match the version of Vivado you are using. UrbanaDDRtest.Tcl

From Vivado, the design can be synthesized, implemented, written to a bitstream, and exported to a .xsa file. To export the bitstream in a .xsa, use "File -> Export -> Export Hardware", check the "include bitstream" option, and provide a file name. An .xsa file will be created, and it can be imported into Vitis and used to define a hardware platform. After the hardware platform is defined, software can be written and executed to exercise the DDR and UART from Microblaze.

To exercise the system, start Vitis, import the .xsa file to create a new hardware platform, build the project, and then create a new application project for the hardware. When creating a new application project, Vitis provides software templates that can be included and used in your design, and one of these templates is a "memory test". This template includes sample code to test the DDR and write to the UART, so the entire system can be exercised without writing any custom code. The Vitis debugger includes a UART terminal program, so the system can be verified without leaving the Vitis environment. To execute the sample memory test code, select the memory test template, build the project, and then use the debugger to run the program. To see the data sent over the UART, start the terminal program before starting the debugger (the UART is only driven at the start of the test).

GPIO

The term "General Purpose Input/Output" (GPIO) generally refers to digital input signals driven by two-state devices like slide switches or pushbuttons, or output signals that drive an indicator device (like an LED). The Urbana board includes 16 single-pole double-throw (SPDT) slide switches, 4 momentary single-pole single-throw (SPST) pushbutton switches, 16 single-color LEDs, and 2 tri-color (RGB) LEDs. An additional 22 GPIO signals drive the Pmod expansion connector.

All LED signals are active high. The individual/discrete LEDs are connected to FPGA pins via 330-ohm resistors, so a logic high signal (3.3V) will illuminate them with about 1mA of current. RGB LEDs have higher voltage thresholds and different current requirements, and so they are driven by transistors. A logic high on any RGB LED signal drives the transistor base and causes about 5mA of current to flow through the LED.

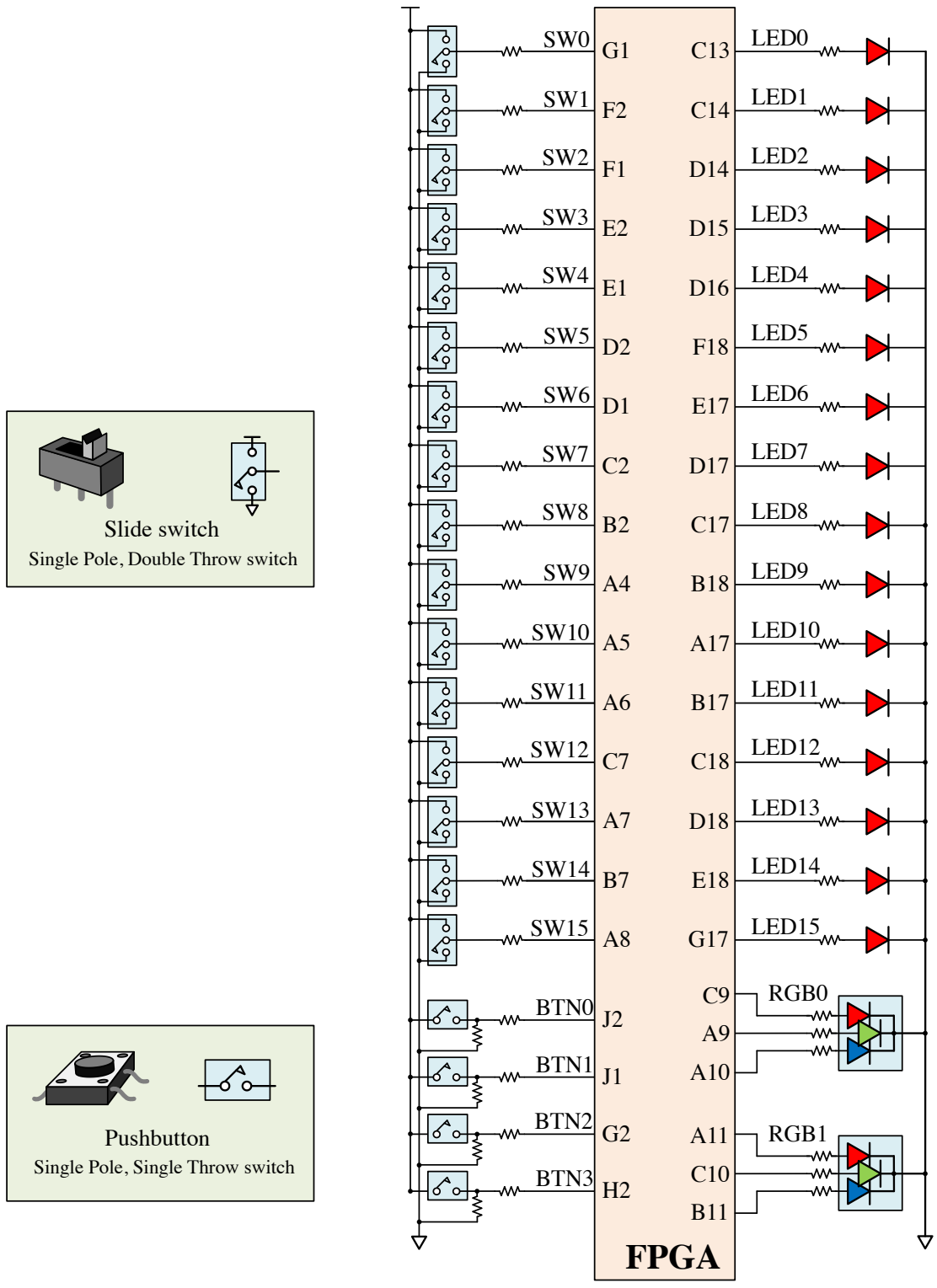


Figure 9. Urbana GPIO devices

Each RGB LED device includes three individual LEDs that are located very close together, giving the appearance of a single LED. Each LED is driven independently, and this allows different colors to be created. By simply turning LEDs on or off, seven different colors can be created. But by controlling the brightness levels of individual LEDs, a broad range of colors can be created. LED brightness levels are typically controlled using pulse-width modulated (PWM) signals. PWM signals are commonly used as simple “digital to analog” converters - when used to control LED brightness, the PWM signals turn individual LEDs on and off at a frequency higher than the human eye can detect, with different lengths of “on” time to create the appearance of different brightness levels. Users can define custom PWM IP blocks to drive the LEDs - you can read more about PWM circuits here: PWM And PDM Signals

Seven Segment Display

A seven-segment display is built from individual LED’s arranged in a figure-8 pattern as shown. Any LED/segment can be individually illuminated, so any one of 128 different patterns can be shown. The figure below shows segment illumination patterns for decimal and hexadecimal digits.

Figure 10. Seven segment display illuminatation patterns

The Urbana board includes two 4-digit seven-segment displays (8 total digits) that use a common anode configuration. Segment LEDs consume about 3mA each (well within the current sourcing capability of the FPGA pins), so the cathodes are tied directly to FPGA pins. Since 24mA+ can flow through the anode signals, the anodes are driven from transistors that can provide the needed current (and the transistors are driven from the FPGA pins). All signals are active low.

To drive a single digit, the corresponding anode signal can be driven (low), and then individual cathodes can be driven (also low) to turn on individual segments. To drive all digits to create an eight-digit display, a scanning display controller is needed. To learn more about seven segment displays, including an example design of a seven-segment controller, see the “Seven segment controller” document: Seven Segment Display

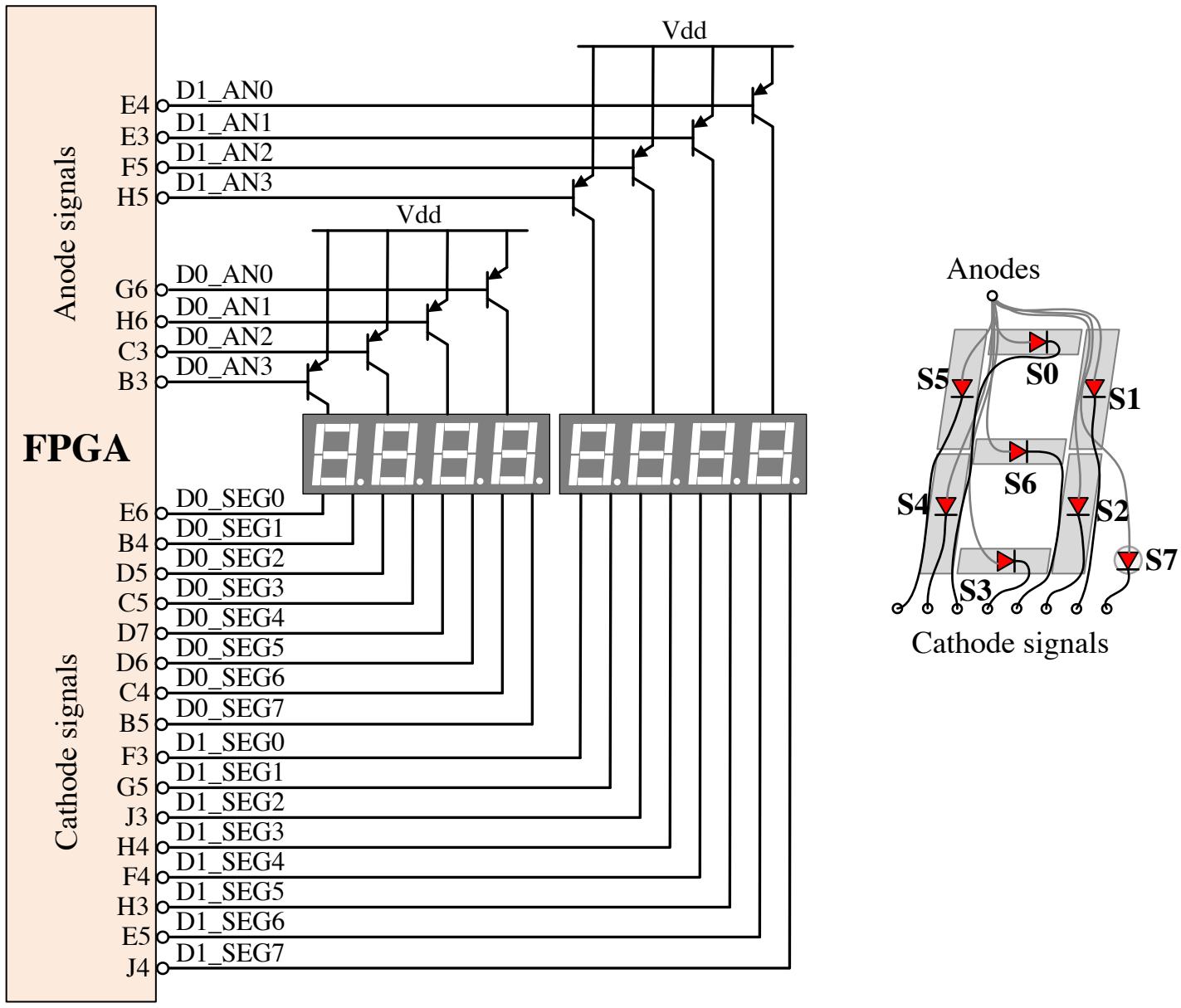


Figure 11. Urbana seven segment display

Clock

The Urbana board includes an Abracon 100MHz 50PPM MEMS/CMOS oscillator that is always running. The oscillator is connected to a multi-region clock-capable (MRCC) FPGA I/O pin (pin N15). From that pin, the clock signal can route to any clock management tile, and/or to any flip-flop in the FPGA using the internal high-speed clock network.

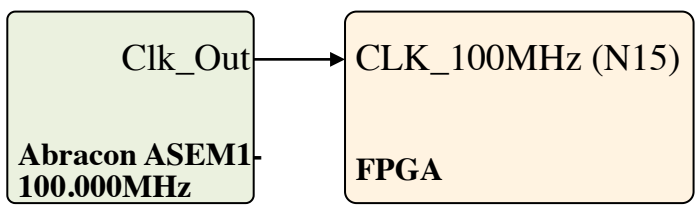


Figure 12. Urbana board clock circuit

accommodate simple square waves or PWM/PDM encoded audio signals. For more on PWM/PDM signals:

PWM And PDM Signals

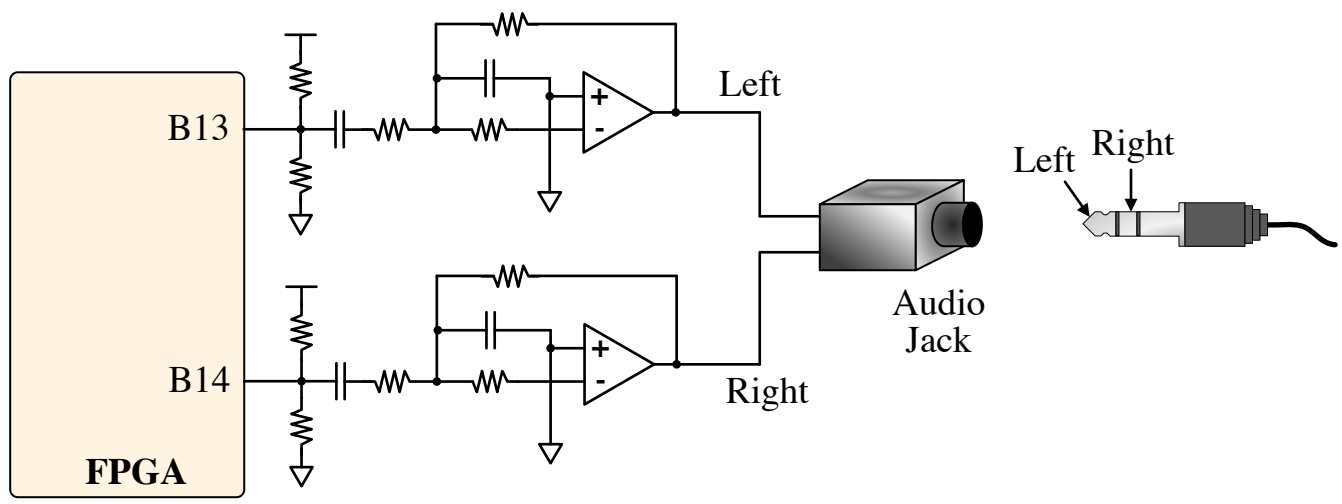
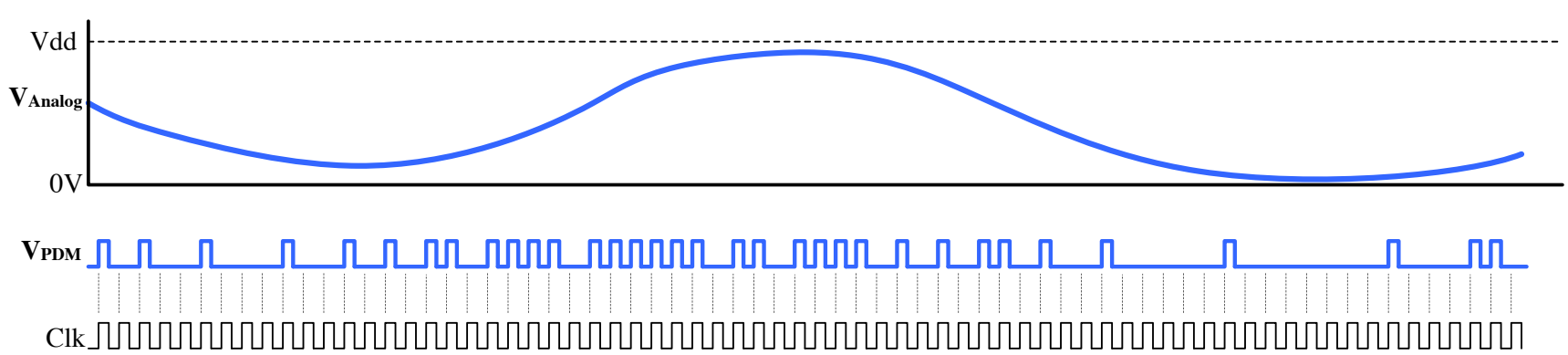


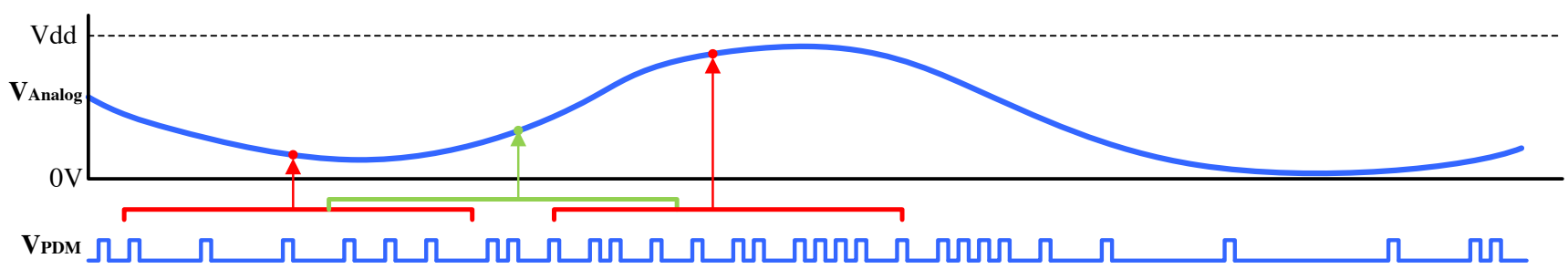
Figure 13. Urbana board audio output circuits

Microphone

The Urbana board includes a TDK T3902 MEMS microphone. The T3902 device, which includes the microphone, amplifier, and sigma-delta modulator, produces a pulse-density modulated signal that can be converted to an n-bit binary sample using an averaging FIR filter implemented in the FPGA. The FPGA produces the clock that drives the T3902, and the T3902 will return pulses on clock edges in proportion to the incident sound pressure on the microphone – a high density of pulses indicates an audio signal near positive full scale, and a low density of pulses indicates a signal near full negative scale.



A constant-width pulse may or may not be produced on any given clock edge. The strength of the incident air pressure waveform (sound loudness) determines the pulse density produced by the T3902



Pulses within a window can be scaled and added to form a binary number (magnitude); windows can overlap any amount to determine samples per second

Figure 14. PDM Waveforms

A FIR filter circuit implemented in the FPGA can count/add the number of pulses over some fixed time window, with different weights applied to pulses at different points in the window, to build a string of binary numbers (samples) that are proportional to audio signal amplitude. The sample values produced by the filter are called “Pulse Code Modulated”, or PCM values. In typical applications, 64 (or more) clocks are driven into the T3902 for each PCM sample. For example, if the FPGA drove a 3.072MHz clock into the T3902, then consecutive groups of 64 points could be combined into a PCM sample, resulting in a 48KHz PCM sample rate (3.072MHz/64 = 48KHz).

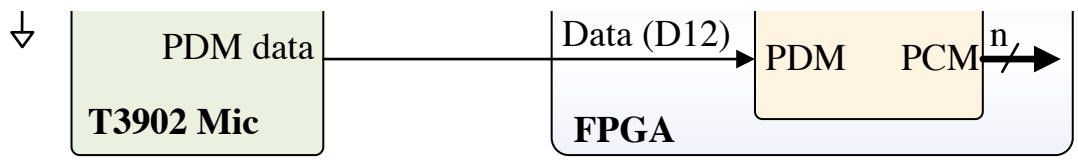


Figure 15. Urbana board MEMs microphone

UART

A “Universal Asynchronous Receiver Transmitter” (UART) is a hardware circuit that is used to send serialized data from one device to another. UARTs use a single wire to move a data, and no timing signal is sent to synchronize the data (that’s where the word Asynchronous in the title comes from). Instead, both the sender and receiver must agree on the data rate (in bits per second, also known as the “baud” rate). UART signals are unidirectional, meaning they always move data in one direction. To move in both directions, two wires are needed – one to transmit data (TXD), and one to receive data (RXD). The Urbana board includes a UART port between the main/programming USB port and the FPGA, and this UART port automatically enumerates on a connected PC as a COM port.

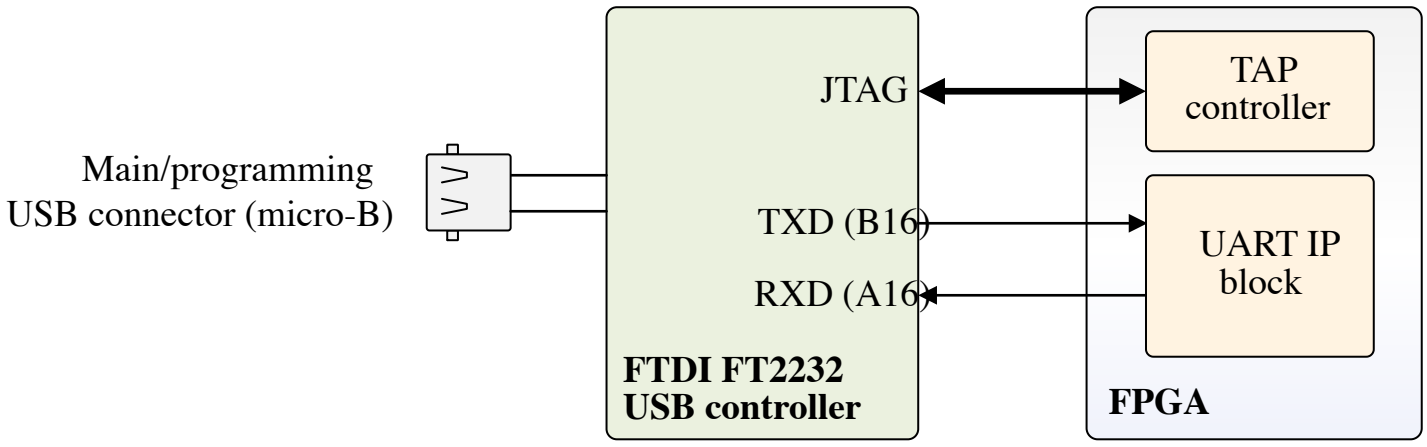


Figure 16. Urbana board UART

A UART core can be designed from scratch, or Vivado’s IP Integrator can be used to instantiate and configure a UART IP block. In a typical use environment, a UART is included as a peripheral circuit for an embedded processor (like Microblaze) to add a simple I/O port for low-bandwidth communications. In this case, the IP Intergator tool can be used to define a UART IP block and connect it to the processor. An example project that uses a UART block in a Microblaze system is avaialble in the DDR section above - you can build this design in Vivado, and then examine the UART block and its configuration parameters.

Bluetooth Radio

The Urbana board includes a Bluetooth 5.0 Low Energy (BLE) radio module based on the advanced Nordic Semiconductor nRF52810 device. The BLE module exchanges data with the FPGA using a simple two-wire UART/serial protocol (RXD/TXD) running at 115,200 bps. Send and Receive data packets can be up to 256 bytes, and are terminated with a newline (\n) character. Data sent from the FPGA is held in a FIFO within the BLE module, and transmitted immediately after a newline is encountered.

The PAIR LED will blink when the BLE radio is ready to be paired to your phone or host computer. Once it is paired, the PAIR LED will be on constantly. If the radio is not paired within one minute, it will enter a deep sleep mode. To wake up the module, press the **BLE RST** located beside the BLE module. A DATA LED near the BLE module will blink whenever data is being actively moved.

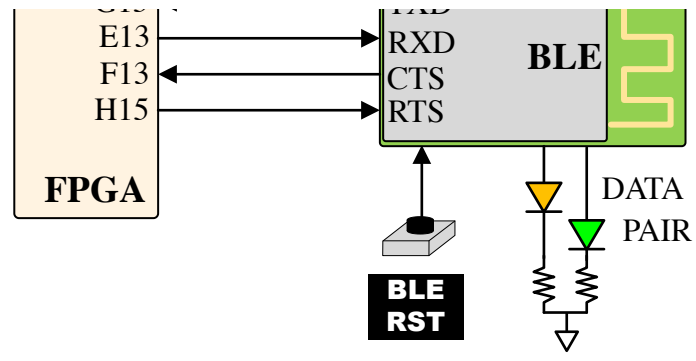


Figure 17. Urbana board BLE radio

HDMI port

The Urbana board includes a TI TPD12S521 HDMI transmitter/buffer and an HDMI video source connector that can drive display resolutions up to 1080P. HDMI is a high-speed digital synchronous interface that uses the TMDS (transition minimized differential signaling) I/O standard. Although it is possible to create an original, custom HDMI controller, it is not trivial. In contrast, a VGA controller is relatively simple to design and implement, and Real Digital provides a VGA to HDMI IP block. This IP block receives the standard VGA signals (HSync, VSync, and VDE (Video Display Enable), and three 8-bit color busses, and converts these signals to HDMI. The IP block can handle VGA resolutions up to 1280x720, and can generate HDMI resolutions up to 1080P.

Note the IP block needs high-speed clocks (faster than the 100MHz clock input to the FPGA). The required clocks can be generated using the FPGA's "clock management tile" (CMT) IP block – a link below offers guidance on configuring and instantiating the CMT IP block. Further details on producing a VGA controller circuit and on accessing and using the VGA-to-HDMI core are available in the links as well.

VGA To HDMI

Analog And Digital Displays

Clocking Wizard

The figure below shows a block diagram of a display controller based on a VGA controller driving the VGA-to-HDMI IP block. In the FPGA block shown in the figure, the VGA Controller is user-designed IP, and the CMT and VGA to HDMI blocks are pre-existing IP blocks that only need to be configured and instantiated. All three blocks can then be assembled in a higher-level HDL module as shown in the example code.

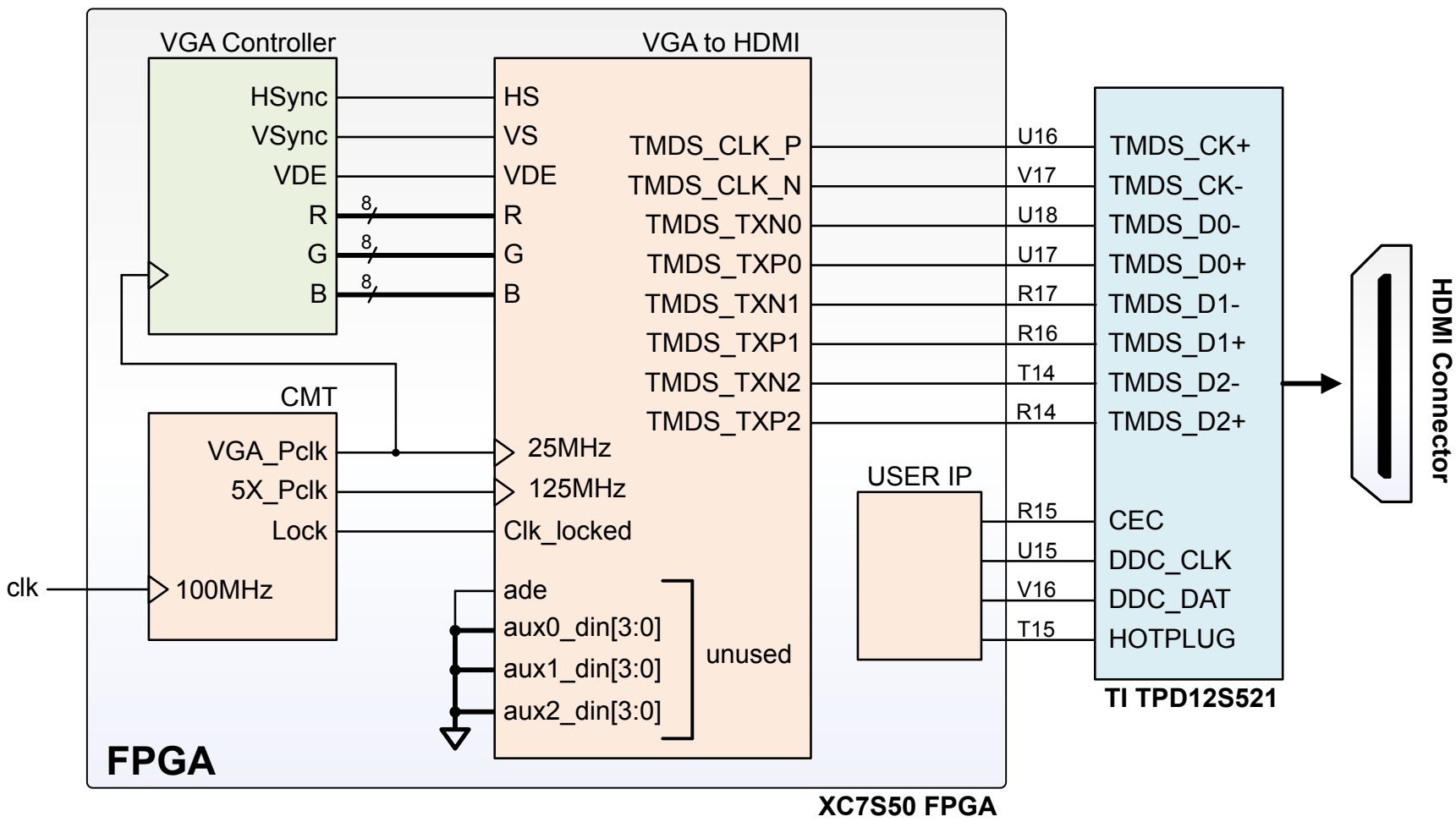


Figure 18. HDMI controller top-level block diagram

sources, including the voltage across two dedicated analog input pins, the voltage across any one of 16 pairs of “accessory analog input” pins (an user-programmable analog multiplexor selects which pair of pins are used), the voltage produced by an on-chip temperature sensor, or the voltage present on any one of several power supply rails. On the Urbana board, a multiturn 10K thumbwheel potentiometer is connected across the two primary ADC inputs (VP is connected to the pot sweeper, and VN is connected to ground).

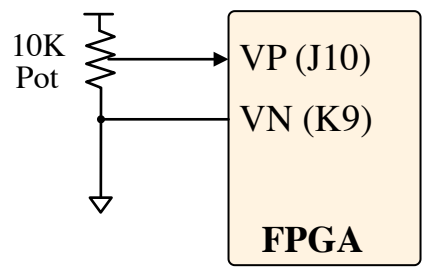


Figure 19. XADC

The XADC outputs are available in several registers as described in AMD’s User Guide 480. The XADC block can be added to a Vivado block diagram and customized using the configuration wizard, or directly instantiated and parametrized in Verilog or VHDL code. Refer to AMD UG480 for information on including the XADC in your design.

Note that if the XADC is not instantiated in a design, then by default it digitizes the output of all on-chip temperature and voltage rail sensors. The most recent of these measurements are stored in dedicated registers for access at any time via the JTAG interface. Further, user-defined alarm thresholds can automatically indicate over-temperature events and unacceptable power supply variation. These user-specified thresholds can be used to initiate an automatic powerdown.

SD Card

The Urbana board includes a micro-SD card socket with the standard SD card signals connected. In a typical use environment, an SD IP block would be instantiated using Vivado’s IP integrator tool, and driven from an embedded processor core like Microblaze.

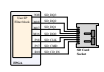


Figure 20. Urbana board SD card socket

USB Host

The Urbana board includes a MAX3421 USB host controller that supports HID devices, mass store devices, and a variety of other devices like game controllers, cameras, and certain cell phones. The MAX3421 handles all USB protocols for communicating with full/low speed USB peripherals, and so relieves the FPGA from having to deal with USB transaction details. Instead, all data exchanged with the USB peripheral is available to the FPGA in a double-buffered 64-byte read FIFO, and a similar but separate write FIFO. A set of 23 8-bit registers in the MAX3421 provides access to the FIFOs and all status and control bits.

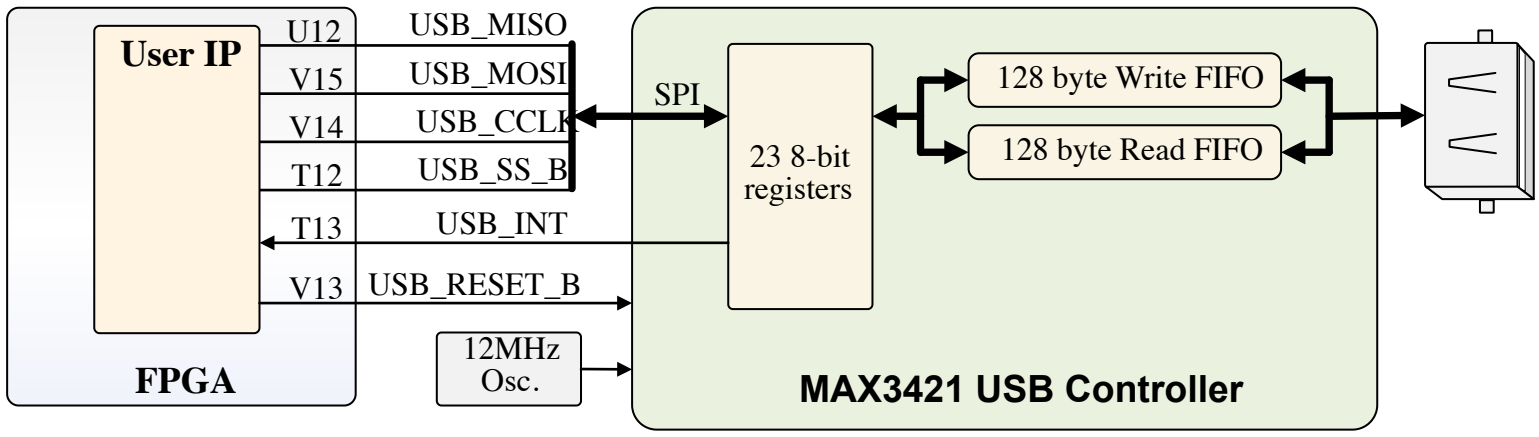


Figure 21. Urbana board USB host

USB Host Shield Github

MAX3421 Data Sheet

Servomotor Connectors

The Urbana board includes four 3-pin connectors that are compatible with standard hobby servo motors. Servo motors can draw up to several hundred milliamps each. Depending on the power available from the USB connector, the Urbana board's main power supply may be able to drive one or two servo motors (note the main power supply may come from the USB cable used to program the board, or from a separate, external USB wall-plug). Servo power routes through jumper J8. To use the main power supply, load a jumper at J8 across the VS/5V pin headers. If more power is needed, a two-pin MTE cable can connect a bench supply to J8 pins VS and GND as shown.

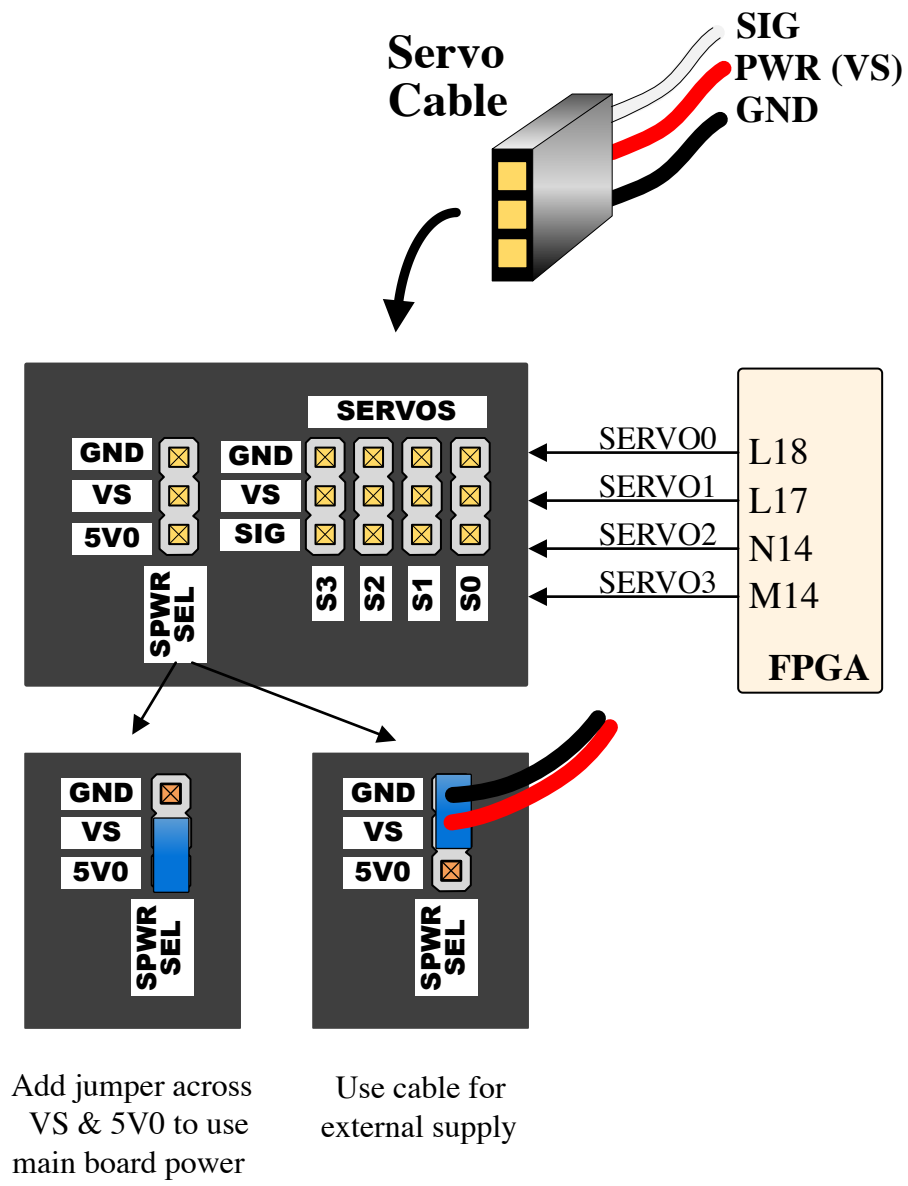


Figure 22. Urbana board servomotor connections

Servomotors are controlled with a single PWM signal that obeys a well-defined protocol (the signal is labeled SIG on the Urbana board's servo connectors). You can learn more about servo motors and their control here: Servomotors

Expansion Connectors

A 30-pin Pmod+ expansion connector makes 22 FPGA signals available on a 100-mil-spaced dual row connector. The 30 connector positions are organized as two standard Pmod ports, plus an 6 additional GPIO signals. The two Pmod ports each include 8 GPIO signals, two ground pins and two Vcc pins as shown, and the 6 additional signals are between the two Pmod ports. All 22 FPGA signals are routed as differential pairs.

The connector signals are organized in this way so that two 12-pin Pmods can be inserted into the marked subsets of holes, or one larger 30-pin connector with 22 GPIO signals can be used.

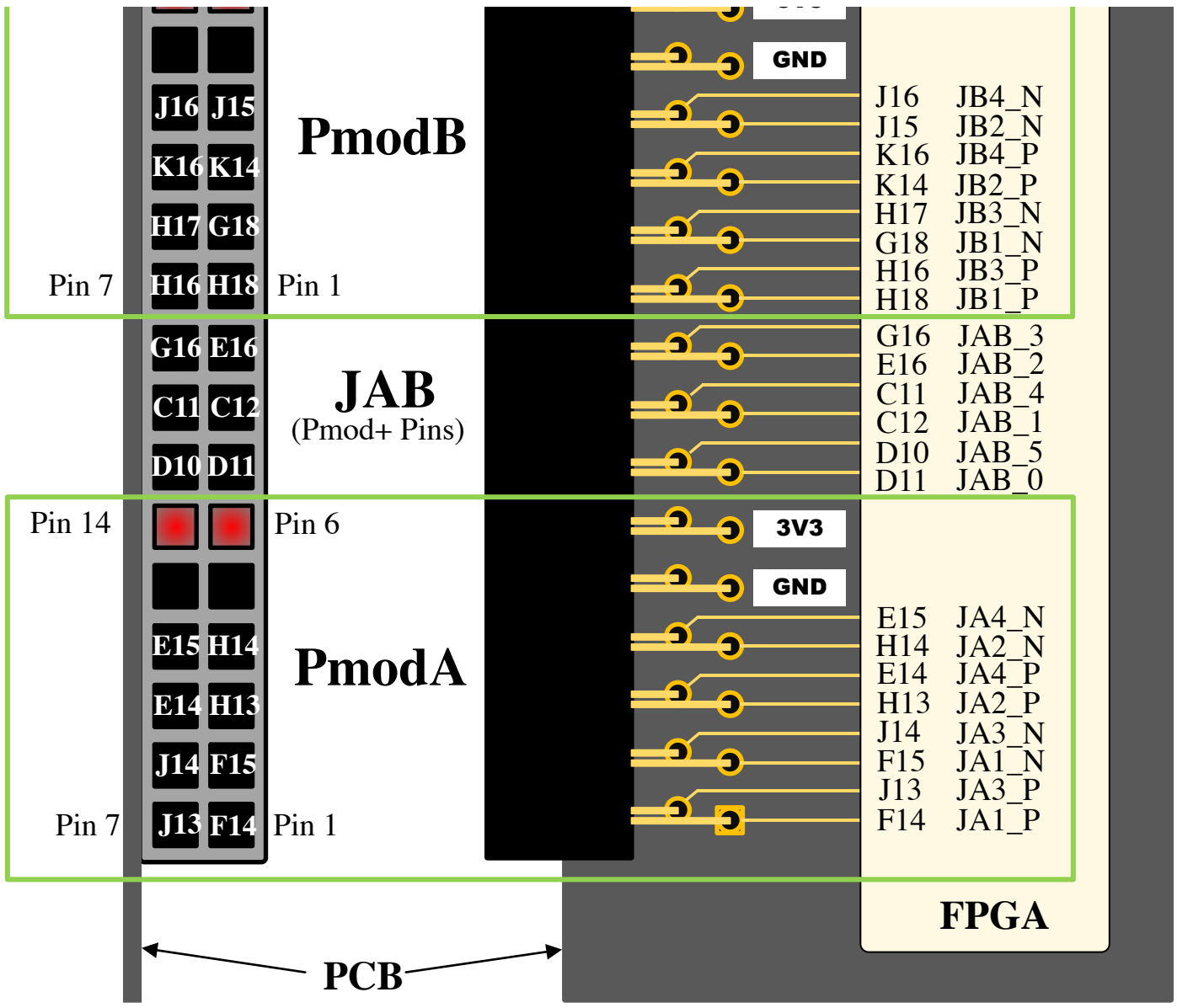


Figure 23. Urbana board Pmod+ connector

Contact Us
contact@realdigital.org
(509) 336 9656
Mon - Fri: 9am - 5pm

Our Location
655 SW James Pl
Pullman WA 99163
USA

Copyright 2025 Real Digital. All rights reserved.